Canvas do Projeto

Seis diagramas da arquitetura do Cinematica — do ETL ao agente, do modelo do grafo ao estado atual.

Cada arquivo é um artboard independente, dimensionado em px (1600 de largura). Eles abrem direto no navegador; o `./support.js` que o cabeçalho referencia pertence ao editor de canvas e dá 404 aqui, sem efeito algum sobre o que é exibido.

01 **Visão geral**
O fluxo completo em dois trilhos: a ingestão offline e o caminho de uma pergunta. `Main.dc.html`

02 **Pipeline de ingestão**
Os cinco estágios do ETL, suas entradas e o que cada um produz. `Pipeline.dc.html`

03 **Pipeline do agente**
Os cinco estágios do LangGraph, o loop de retry e as três saídas do `verify`. `Agente.dc.html`

04 **Modelo do grafo**
Os nove rótulos de nó e as relações que os ligam. `Grafo.dc.html`

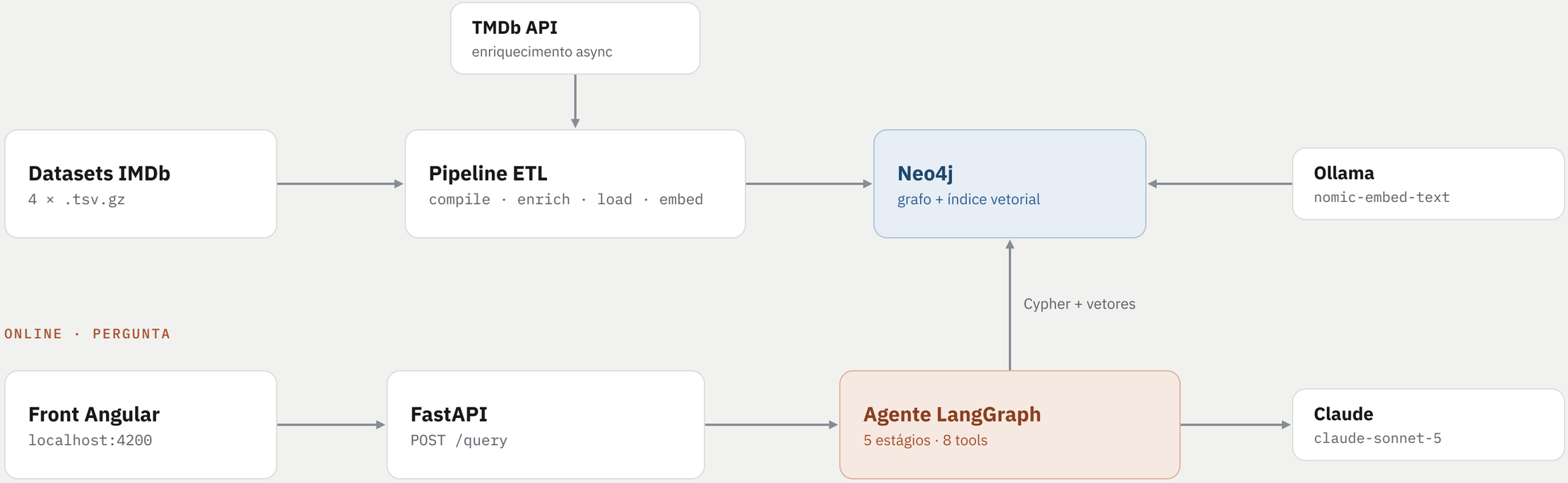
05 **Grounding tools**
As oito ferramentas expostas ao modelo, entre leitura e escrita. `Tools.dc.html`

06 **Estado do projeto**
O que está verificado, o que nunca rodou e o que ficou em aberto. `Estado.dc.html`

Cinematica

Knowledge graph de cinema com camada de agente. IMDb + TMDb viram grafo no Neo4j; o Claude raciocina sobre ele através de ferramentas de grounding.

OFFLINE · INGESTÃO



~5.000
linhas de Python

~2.200
linhas de TypeScript

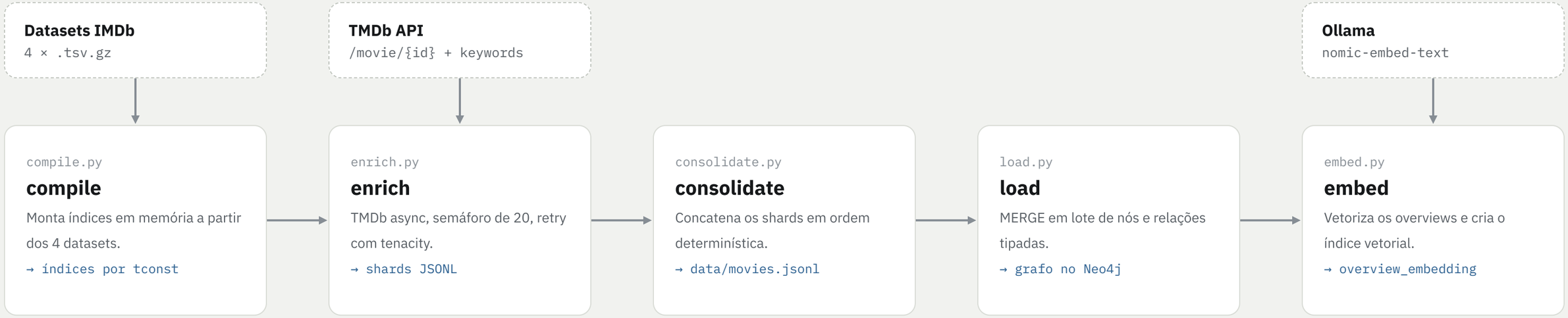
9
tipos de nó no grafo

8
grounding tools

5
estágios de raciocínio

Pipeline de ingestão

Offline, rodado por `python -m src.main`. Nada é carregado inteiro na memória: os datasets são lidos linha a linha.



Retomável

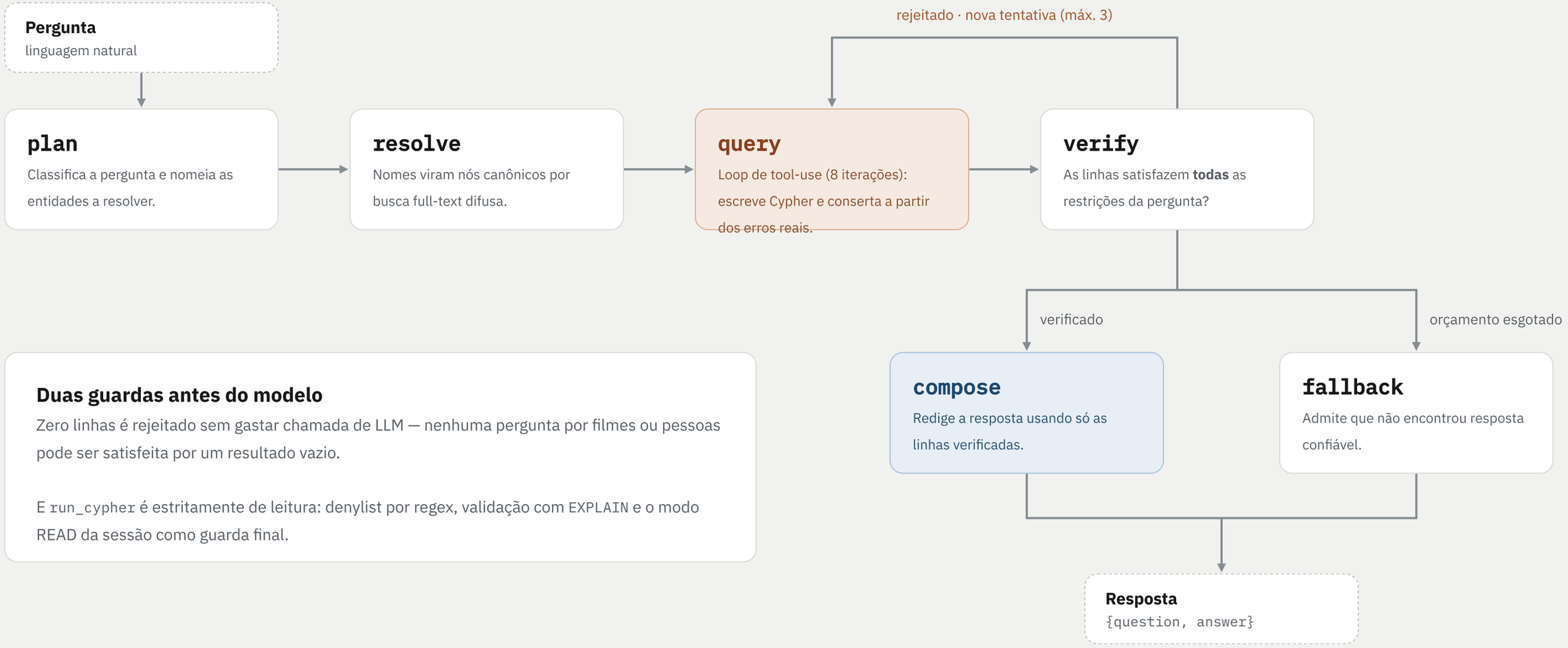
Os shards são nomeados a partir dos `tconst` ordenados, então um chunk sempre mapeia para os mesmos filmes. Shards que já existem são pulados — dá para rodar em lotes com `python -m src.main 200` e continuar depois.

Idempotente

Toda escrita no Neo4j é `MERGE`, e o embed só busca filmes que ainda não têm vetor. Reexecutar sobre a mesma entrada — ou sobre uma que cresceu — não duplica nada nem refaz trabalho.

Pipeline do agente

Cinco estágios em LangGraph, cada um uma chamada ao Claude. Nenhuma resposta é composta a partir de dados não verificados.



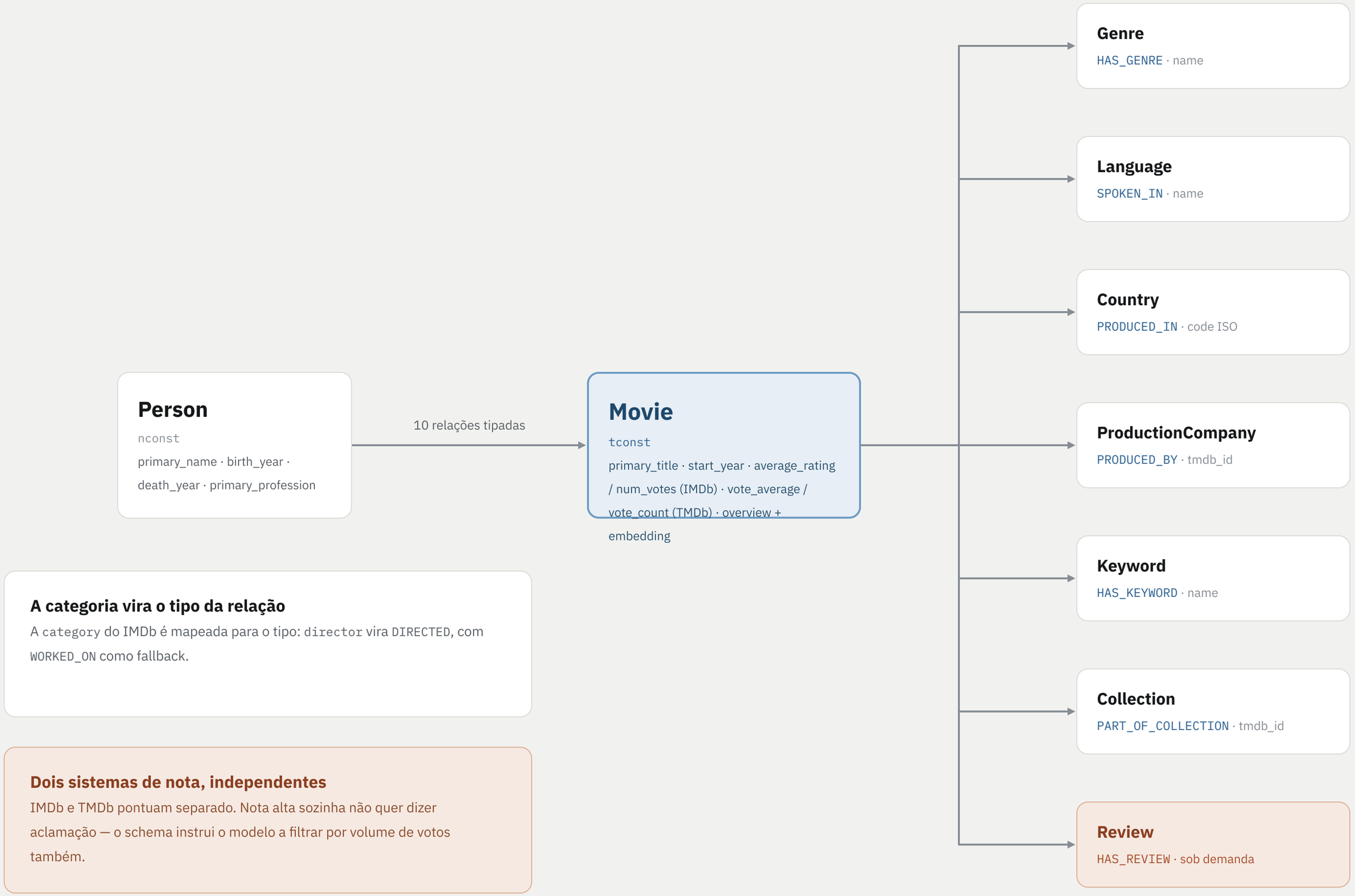
Duas guardas antes do modelo

Zero linhas é rejeitado sem gastar chamada de LLM — nenhuma pergunta por filmes ou pessoas pode ser satisfeita por um resultado vazio.

E `run_cypher` é estritamente de leitura: denylist por regex, validação com `EXPLAIN` e o modo READ da sessão como guarda final.

Modelo do grafo

Nove rótulos de nó. Cada dimensão vira nó de primeira classe, nunca propriedade solta no filme.



A categoria vira o tipo da relação

A `category` do IMDb é mapeada para o tipo: `director` vira `DIRECTED`, com `WORKED_ON` como fallback.

Dois sistemas de nota, independentes

IMDb e TMDb pontuam separado. Nota alta sozinha não quer dizer aclamação — o schema instrui o modelo a filtrar por volume de votos também.

Grounding tools

Oito ferramentas expostas ao Claude durante o estágio `query`. Quatro só leem o grafo; quatro escrevem nele sob demanda.

LEITURA

`get_schema`

Devolve o schema real do grafo. Chamada antes de escrever Cypher, para o modelo não adivinhar nomes de propriedades.

`resolve_entity`

Nome livre vira nós candidatos por full-text. Tolera caixa e grafia, onde o match exato falharia em silêncio.

`run_cypher`

Roda Cypher de leitura. O erro do Neo4j volta estruturado, como insumo para o modelo se corrigir.

`vector_search`

Filmes semanticamente próximos de um tema, para perguntas que Cypher não expressa.

ESCRITA · SOB DEMANDA

`search_external_movie`

Procura no TMDb um filme ausente do grafo e devolve candidatos com o `tmdb_id`.

`ingest_movie`

Traz esse filme para o grafo. No-op se ele já existir, para nunca sobrescrever o registro vindo do IMDb.

`fetch_reviews`

Busca as reviews públicas do TMDb para o próprio modelo ler e julgar.

`save_movie_sentiment`

Persiste o veredito do modelo: nós `Review` mais o agregado no `Movie`.

A escrita nunca passa pelo Cypher. `run_cypher` rejeita qualquer cláusula de escrita por denylist, valida com `EXPLAIN` antes de executar e roda no modo READ da sessão — a guarda autoritativa. As duas rotas de escrita são as ferramentas dedicadas acima.

Estado do projeto

Em 9 de outubro de 2026, no branch `feature/front-openui`, um commit à frente da `main`.

VERDE

Verificado

45 testes de backend passando

pylint 10.00/10 em `src/`

Front: build limpo e **9 testes** no Vitest

Todo o código roda em **Python 3.11**, sem usar nada de 3.12+

AMBIENTE

Nunca executado de verdade

`logs/cinematica.log` só tem saída de `pytest` — o pipeline nunca viu dado real

`datasets/` tem só o README; faltam os 4 `.tsv.gz`

`.env` só com as duas chaves do TMDb, ambas vazias

Sem `NEO4J_*` e sem `ANTHROPIC_API_KEY`; Docker e Ollama parados

ABERTO

Pendências conhecidas

O `make api` não funciona. O `pyproject` exige 3.14, o venv é 3.11 — subir pelo `.venv/bin/uvicorn`

O branch cita **Open WebUI**, que foi descartado no planejamento

A coleção Bruno chama `GET /question`; a rota real é `POST /query`

Para a primeira execução real, nesta ordem

01

Preencher o `.env`: credenciais do Neo4j e a chave da Anthropic

02

Baixar os datasets do IMDb para `datasets/`

03

Subir a infra: `make up` e `make ollama-up`, depois ingerir em lotes

04

API pelo `.venv/bin/uvicorn` e front com `make front`